

A Major Project Report

On

“SPAMMER DETECTION IN SOCIAL NETWORKS USING DEEP LEARNING”

Submitted to

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, HYDERABAD

In partial fulfilment of the requirement for the award of Degree of

BACHELOR OF TECHNOLOGY

In

“Computer Science & Engineering (AI&ML)”

By

Saarah Sultana [228P1A6629]

Under the guidance of

Mrs. Vidya Prathima

Assistant Professor, C.S.E(AI&ML)

Department of Computer Science & Engineering



AAR MAHAVEER ENGINEERING COLLEGE

(Affiliated to JNTU Hyderabad, Approved by AICTE)

Vyasapuri, Bandlaguda, Post: Keshavgiri, Hyderabad-500005

2025-2026



ESTD : 2010

AAR MAHAVEER ENGINEERING COLLEGE

Vyasapuri, Bandlaguda, Post: Keshavgiri, Hyderabad- 500005

(Approved by AICTE, Affiliated to JNTU, Hyderabad)

(Sponsored by Mahaveer Educational Society)

EAPCET/ECET Counselling Code: **AARM** JNTUH College Code: **8P**

CERTIFICATE

This is to certify that the major project work report entitled “**SPAMMER DETECTION IN SOCIAL MEDIA USING DEEP LEARNING**” which is being submitted by **Saarah Sultana [228P1A6629]** in partial fulfilment for the award of the Degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE & ENGINEERING (AI&ML)** of **JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY**, is a record of the bonafide work carried out by them under our guidance and supervision.

Mrs. Vidya Prathima

Assistant Professor.

Mrs. Madiha Samreen

Project Co-ordinator,
Professor & HOD

External Examiner

Dr. S. Sudhakara Reddy

Professor & Principal

DECLARATION

I hereby declare that results embodied in the project entitled “**SPAMMER DETECTION IN SOCIAL MEDIA USING DEEP LEARNING**” has been carried out by us together during the academic year 2025- 2026 as a partial fulfillment of the award of the Bachelor of Technology degree in **COMPUTER SCIENCE & ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)** from Jawaharlal Nehru Technological University, Hyderabad. We have not submitted it to any other university or organization for award of any other degree.

Saarah Sultana

[228P1A6629]

9121874662

ACKNOWLEDGEMENT

I would like to express our deep felt appreciation and gratitude to **Mrs. VIDYA PRATHIMA, Assistant. Professor, Department of C.S.E(AI&ML)**, our project guide, for his/her skilful guidance, constant supervision, timely suggestion, keen interest and encouragement in completing the individual seminar within the stipulated time.

I wish to express our gratitude to **Mrs.MADIHA SAMREEN, Project Co-ordinator, Professor & Head of Department of CSE[AIML]** ,who has shown keen interest and even rendered his valuable guidance in terms of suggestions and encouragement extended to me with an immense care and zeal.

I express our thanks to **Dr. S. SUDHKAR REDDY** , Principal of our college and the management of Mahaveer Institute of Science & Technology for providing excellent academic environment in the college.

I wish to express our gratitude to the **Members of staff** and all others who helped us in more than one way. I would also like to thank the **Lab assistants and Programmers** for helping us through our project.

Saarah Sultana [228P1A6629]

ABSTRACT

Online social networks (OSNs) are platforms for connecting and interacting with people who have similar values and interests. In recent years, social media networking sites like Twitter, Facebook, Myspace, Instagram, and LinkedIn have become extremely popular. As is well known, when compared to other social media platforms, Twitter is one of the most well-known and significant networking sites. It is possible for a user to engage in undesired behaviour or transmit spam messages during communication to obstruct the flow of the conversation. Because more spam messages are being sent online despite filtering mechanisms, this is causing an increasing problem with online spam. As a result, spam classification has grown more difficult as a result of the intricacy that spammers impose on their messages. It is challenging to identify these spam texts. Literature indicates that there is a lot of research being carried out to find and combat malicious accounts and spammers over the last two decades. Hence, various methods have been developed to filter spam. In this project, the methodologies used are deep learning algorithms such as SVR, Random Forest, and Decision tree. In the proposed mechanism, we apply these algorithms on real datasets of Twitter to detect spam messages. Here the spam is divided into different spammers, such as positive spam, negative spam, moderate spam, etc. The performance metrics like precision, recall, F1-score, and support are used to measure the execution of the proposed mechanism.

TABLE OF CONTENTS

NAME OF TOPIC	PAGE NO
Chapter 1. INTRODUCTION	1-6
1.1 SDS: Social Spammer Detection With Sentiment Information	
1.2 Sina Weibo	
1.3 Behavior detection approaches	
1.4 Decision tree and KNN classifier	
1.5 Ontology Based Framework For Criminal Intention Classification (OFCIC)	
1.6 Learning From Unlabeled	
1.7 Asymmetric Self-Learning Approach	
1.8 Hybrid Approach	
1.9 Convolutional Neural Network	
1.10 Deep Neural Network	
1.11 Long Short Term Neural Networks	
1.12 Spam Detection Techniques	
Chapter 2. LITERATURE SURVEY	7-9
Chapter 3. DEEP LEARNING BASED SPAMMER DETECTION	10-24
3.1 Modules	
3.2 Deep learning Methodology	
3.2.1 Deep learning working	
3.3 SVR CLASSIFIER.	
3.3.1 Working Steps of SVR	
3.4 NLP Library file	
3.4.1 Accessing and Processing	
3.4.2 Loading of NLP files	

3.5 Random Forest Classifier

3.5.1 Reasons for using Random Forest 30

3.5.2 Working of Random Forest

3.6 Decision tree Classifier

3.6.1 Reasons for using the Decision tree

3.6.2 Decision Tree Terminologies

3.6.3 Working of decision tree algorithm

Chapter 4. Design

25-30

4.1. Architecture

4.2. UML Diagrams

4.2.1 Use Case Diagram

4.2.2 Activity Diagram

4.2.3 Class Diagram

4.2.4 SequentialDiagram

Chapter 5. RESULTS AND DISCUSSIONS

31-41

5.1 Hardware and Software Requirements

5.2 Software Environment

5.3 Outputs of the project

5.4 Evaluate Models Using Metrics

5.5 Tables and Graphs

Chapter 6. FUTURE SCOPE

42

Chapter 7. CONCLUSION

43

Chapter 8. REFERENCES

44-47

LIST OF FIGURES

S.NO	Figure no	Figure Name	Page No
1.	Figure3.1	Modules flow chart	13
2.	Figure3.2	Deep learning layers	14
3.	Figure3.3	Working flow chats of deep learning	15
4.	Figure3.4	Flow of Datasets	20
4.	Figure3.5	Process of Validation set	20
5.	Figure3.6	Iteration moments of datasets	21
6.	Figure3.7	Working process of Random forest	22
7.	Figure3.8	Working process of Decision tree	24
8	Figure4.1	Archietcture	26
9.	Figure4.2.1	Use Case Diagram	27
10.	Figure4.2.2	Activity Diagram	28
11	Figure4.2.3	Class Diagram	29
12.	Figure4.2.3	Sequential Diagram	30
13.	Figure5.3.1	Output 1	34
14.	Figure5.3.2	Output 2	35
15.	Figure5.3.3	Output 3	35
16.	Figure5.5.1	Comparision graphs of Accuracy and Precision	39
17.	Figure5.5.2	Comparision graphs of Recall, F1-score and Support	40
18.	Figure5.5.3	Comparison of accuracy of algorithms	40

LIST OF TABLES

TableNo	Table Name	Page No
4.1	Comparison of accuracy and precision	38
4.2	Comparison of recall F1-score and support.	39



CHAPTER 1

INTRODUCTION

Online social networks (OSNs) serve as a forum for interaction and connection amongst people who share similar values and interests. OSN platforms are visited by users to gather information that is pertinent to them and to create social and professional networks. OSNs like Facebook, Twitter, and LinkedIn are utilised by millions of people throughout the world to promote interpersonal ties, and their user bases are growing daily. These OSNs are evolving into a fresh platform for the communication of news, opinions, and information. While this is going on, certain users referred to as spammers are abusing these OSN platforms and disseminating false information, propaganda, rumors, fake news, unsolicited communications, etc. Spammer detection has been studied for years in traditional platforms such as email and the Web, which differ substantially from social media services. First, social media services allow users to easily connect with each other, sometimes without mutual consent. For example, in Twitter, anyone can follow anyone else without prior consent from the follower. Many users simply follow back when they are followed by someone for the sake of courtesy. Understanding and detecting deception has been extensively studied in psychology and social sciences. It is well established that micro-expressions play a distinct role in detecting deception. Micro-expression is an involuntary facial expression of humans according to sentiments experienced. It usually occurs when a person is consciously trying to conceal all signs of how he or she is feeling.

Spammer Detection:

Spammer detection on various platforms, e.g., email and the Web, have been studied for years. The spams are designed to corrupt the user experience by spreading ads or driving traffic to particular web sites. A popular and well-developed approach for anti-spam applications is learning-based filtering. The basic idea is that we extract effective features from the labeled data and build a classifier. Then classify new users messages as either spam or ham according to their content information.

Sentiment analysis on product reviews has been a hot topic for quite a few years. Recently, the opinion-rich resources in social media attracted attention from disciplines. As an effective tool to understand opinions of the public, sentiment analysis is widely applied in various social media applications, including poll rating prediction, event prediction, etc. Detection of Spammers over one of the most popular OSN platforms; Twitter. Twitter is the one of popular and largest networking sites compare to other social media sites. Twitter has been allows social media networking users to post latest news and share messages. The size of the posted messages is no more than 280 characters; such messages are called tweets in Twitter network. Social media thus is an evolution of the Internet, where people connecting themselves with the world. The most important types of social media span are, Bookmarking sites, Blogs, RSS Feeds, Linking and posting, Micro blogs Content Rating, Widgets, Audio podcasting and Social networking,



podcasts of video. An individual can sign up for an account on a social networking website to establish their online persona. They can then choose which other users they want to connect with and interact with on the platform by using an API interface. Contacts, location, associations, personal information, employment history, personal preferences, friends, and other information are all included in "the information a social network obtains about a user." Eighty-two percent of the world's population uses social media once a week, with 48 percent of users engaging daily. The majority of the time, people use social networking sites to share their feelings, ideas, and beliefs about other people as well as products. Now a days 0.13% of messages advertised on Twitter are clicked, whenever users click on these links they accessed into spam data, which are higher than email spams . Features of Twitter. Twitter users may "follow" other accounts that interest them. In contrast to other social media platforms, which employ unidirectional linkages, the interaction between users on this one is bi-directional, therefore a user may not necessarily be following a person who is one of his followers. In order to share a tweet with his "following," a user can "like" or "re-tweet" (RT)" it. The interaction between Twitter users, where each user has a distinct Twitter username and users can write tweets mentioning other users by including their usernames with the opening "@" character. When someone mentions, likes, or RTs one of his tweets, users immediately notified through alerts. How Twitter Deals with Spam. In order to maintain a spam-free environment, Twitter utilises both manual and automatic services to combat spammers. Twitter allows users to manually report spammers using the spammers' profile pages . Twitter uses a number of different factors, such as posting identical messages on multiple accounts or multiple identical messages on one account, following/unfollowing a large number of accounts quickly, having a high number of spam complaints lodged against the account, aggressively liking, following, and retweeting, posting malicious links, posting tweets that primarily consist of links rather than personal updates, and posting unrelated tweets to a topic. Preprocessing was then applied to the Twitter dataset to get a normalised set of features based on which the activities of positive, negative and moderate spammers were studied. The key features extracted were Followers/Followes, URLs, Spam Words, Replies and Hash Tags. Following the acquisition of these characteristics, three conventional learning algorithms, namely SVR, Random forest, and Decision trees, were applied to them in order to detect Spammers in the dataset. Finally, the improvement in spam detection is quantified using accuracy metrics, and the findings reveal that our innovative integrated strategy, which integrates all algorithms, beats other traditional techniques in regards to both overall accuracy and non-spammer detection accuracy.

1.1 SDS: SOCIAL SPAMMER DETECTION WITH SENTIMENT INFORMATION:

The performance of SDS is better than the four baselines, which are based on different strategies of utilizing content and network information. This demonstrates that the integration of sentiment information positively helps improve social spammer detection performance. Using user-based features and content-based features to facilitate spam detection. The performance of four traditional classifiers, namely Random Forest, Support Vector Machine, Naïve Bayesian and K-Nearest Neighbor classifiers, in their abilities to distinguish suspicious users from normal ones content information, one way is to learn a supervised model, and apply the learned model for spammer detection. Due to the unstructured and noisy content information in social media, this method yields two problems to be directly applied to our task. First, text representation



models Second, In addition to the short form of texts, method is built on a non-negative matrix factorization model (NMF) (Lee and Seung 1999), which seeks a more compact but accurate low-rank representation of the users .

1.2 SINA WEIBO :

The spammer feature to detect spammer and test the results over SinaWeibo, the biggest social network site in China. Under the Weibo API, a specific dataset crawler is developed to extract any unauthorized users public messages inside the Weibo platform. features related to message content and user behavior and apply the SVM based classification algorithm for spammer detection .The SVM with RBF kernel function has two such training parameters: C controls over fitting of the model; and gamma controls the degree of nonlinearity. In the experiment, they applied a parameter selection tool provided by lib SVM to select parameters automatically with a5-foldcross-validation. This tool uses grid search policy to find highest classification accuracy through computation from different values of C and gamma pair .Weibo has become an important way to gain information and make statements. While it also creates a fast growing number of spammers, who tend to publish tendentious remarks which confuses the public and the authenticity of information. The Weibo spammers are employed in latent network public relations and marketing companies, who-se job is to achieve publicity or vilify the effect by means of organizing and casing a focus of speculation to a topic or person. They try to guide the public opinion on Weibo, so that people's real needs cannot be expressed, which leads to 'one-sided' speech. The spreading of spam degrades user experience and also negatively impacts server-side functions such as data mining, user behavior analysis, and resource recommendation.

An extreme learning machine (ELM)-based supervised machine is proposed for effective spammer detection. A set of features is then extracted from message content and user behavior and applies them to the ELM-based spammer classification algorithm.

1.3 BEHAVIOR DETECTION:

For behavior detection approaches including Bayesian algorithm , decision tree , k-means and logistic regression algorithm, they use some features as attributes. However, it's difficult to adapt to the changing and self-disguise of new Weibo spammers. Besides, another drawback of these methods is the lack of a clear threshold to distinguish between spammers and legitimate users. A group of behavior features are set to build a classification model for detecting spammer on Weibo. To build a classifier with higher precision, our study innovatively combines Bayesian Algorithm (which can simultaneously return higher Spammer Recall and Legitimate Recall) and Genetic Algorithm (to create an optimal threshold matrix which can efficiently increase the Precision of the model by improving the conditional probability matrix). From the result, it is easy to see that classifier has a 1298 good learning capacity, which shows that it can identify spammers on Weibo effectively. It can also be used to classify Weibo users in practice after training by several latest samples.



1.4 DECISION TREE AND KNN:

CLASSIFIER Decision Tree is a Supervised learning technique that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where internal nodes represent the features of a dataset, branches represent the decision rules and each leaf node represents the outcome. In a Decision tree, there are two nodes, which are the Decision Node and Leaf Node. Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches. The decisions or the test are performed on the basis of features of the given dataset. K-nearest neighbors (KNN) algorithm is a type of supervised ML algorithm which can be used for both classification as well as regression predictive problems. However, it is mainly used for classification predictive problems in industry. The following two properties would define KNN well Lazy learning algorithm -KNN is a lazy learning algorithm because it does not have a specialized training phase and uses all the data for training while classification. Nonparametric learning algorithm -KNN is also a non-parametric learning algorithm because it doesn't assume anything about the underlying data.

1.5 ONTOLOGY BASED FRAMEWORK FOR CRIMINAL INTENTION CLASSIFICATION (OFCIC):

The Ontology-Based Framework for Criminal Intention Classification (OFCIC). The solution explores automatic classification models and algorithms applied to short textual messages to help in the detection of digital criminal acts. We propose the Ontology of Criminal Expressions (OntoCexp) to provide a formal and extensible model for representing CSE on social networks. The framework OFCIC uses the ontology OntoCexp for selecting potentially crime related posts, as well as to automatically decipher the posts. The ML techniques are used for automatically classifying the posts according to an intention classification framework. Our solution provides computational mechanisms and a software prototype to support investigators in the task of selecting potential criminal posts and filter them according to a predefined intention classes from SAT. This work contributes and differs from other studies by proposing a hybrid framework, which uses fundamentals, concepts, and techniques from Semantic Web, Semiotics, SAT, and ML[19]. ANN - An artificial neuron network (neural network) is a computational model that mimics the way nerve cells work in the human brain. Artificial neural networks (ANNs) use learning algorithms that can independently make adjustments - or learn, in a sense - as they receive new input. This makes them a very effective tool for nonlinear statistical data modeling.



1.6 LEARNING FROM UNLABELED -Lfun :

The ‘pieces of data that have not been tagged with labels identifying characteristics, properties or classifications’. Unlabeled data has no labels or targets to predict, only features to represent them. Think of a list of emails without tags that mark them as ‘spam’ or ‘not spam’. Or a set of images without identifiers like ‘people’, ‘cars’, ‘animals’, etc. So, unlabeled data is basically raw data that has not been annotated by human experts. However, unlabeled data can be quite effective for machine learning. It is mostly used for unsupervised learning (aka exploratory data analysis). In a similar way, labeled data allows supervised learning where label information about data points supervises any given task.

1.7 ASYMMETRIC SELF-LEARNING APPROACH:

An asymmetric self-learning (ASL) approach. The proposed ASL can discover new information of changed tweeter spam and incorporate it into classifier training process. A number of experiments are performed to evaluate the ASL approach. The results show that the ASL approach can be used to significantly improve the spam detection accuracy of using traditional ML algorithms.

1.8 HYBRID APPROACH:

Hybrid techniques are a combination of two or more computational techniques which provide more advantage to detect components than any other individual technique. It helps to improve data analysis. The hybrid technique helps qualitative research to be effective. It combines various methods to produce recommendations in order to combine the robustness and eliminate the drawbacks of the individual techniques .

1.9 CONVOLUTIONAL NEURAL NETWORK:

A convolutional neural network, or CNN, is a deep learning neural network sketched for processing structured arrays of data such as portrayals. CNN are very satisfactory at picking up on design in the input image, such as lines, gradients, circles, or even eyes and faces. The agenda for this sphere is to activate machines to view the world as humans do, perceive it in a alike fashion and even use the knowledge for a multitude of duty such as image and video recognition, image inspection and classification, media recreation, recommendation systems, natural language processing, etc .

1.10 DEEP NEURAL NETWORK:

A deep neural network is a neural network with a certain level of complexity, a neural network with more than two layers. Deep neural networks use sophisticated mathematical modeling to process data in complex ways. The agenda for this sphere is to activate machines to view the world as humans do, perceive it in a alike fashion and even use the knowledge for a multitude of duty such as image and video recognition, image inspection and classification, media recreation, recommendation systems, natural language processing



1.11 LONG SHORT TERM NEURAL NETWORKS:

Long Short Term Memory is a kind of recurrent neural network. In RNN output from the last step is fed as input in the current step. LSTM was designed by Hochreiter & Schmidhuber. It tackled the problem of long-term dependencies of RNN in which the RNN cannot predict the word stored in the long-term memory but can give more accurate predictions from the recent information. As the gap length increases RNN does not give an efficient performance. LSTM can by default retain the information for a long period of time. It is used for processing, predicting, and classifying on the basis of time-series data .

1.12 SPAM DETECTION TECHNIQUES:

Many efforts have been made to develop spam detection techniques on Twitter in the last decade. Detection based on Syntax Analysis Syntax-based detection methods analyses tweets at character or word level. There are some work focusing on inspecting shortened URLs inside tweets. Shortened URLs can be generated by shortener services by inputting a long URL, which are used by spammers to hide their malicious URLs. Lee and Kim Detection based on Feature Analysis, In this field, there are also many methods that select account and/or tweet features as training data for the input of machine learning based classifiers, such as the work . Blacklist Techniques are commonly deployed in web filtering services such as Twitter spam detection, with the functionality of blocking malicious websites according to their information analysis like user feedback and website crawling. Ma, et al. presented a lightweight blacklisting approach with lower cost than existing classifiers . Twitter spam detection techniques, and proposed a new classification method based on deep learning algorithms to address them. For the purpose of judging its performance evaluation, we firstly collected a part of labeled data (376,206spam and 73,836 nonspam tweets) from a 10-day ground truth dataset with more than 600 million realworld tweets. The suggested integrated strategy, which incorporates three learning algorithms, namely SVR, Random forest, and Decision trees, is the main contribution of our study, with the aim of enhancing spam detection accuracy.



CHAPTER 2

LITERATURE SURVEY

Online social networks (OSNs) are flooded with social spammers. They appear in various forms such as fake accounts, fake reviews, malicious links, and bulk messages. Social spammers take advantage of OSN platforms to publish malicious content, spread phishing scams and perform promotional campaigns. Social bots are very popular and highly influential in OSNs. But their honesty is questionable where people deploy social bots for both genuine and malicious purposes. They further investigated that, a very little amount of work is published on bot detection and detecting harmful behaviors in OSNs. De Paoli investigated the importance of detecting social bot behaviors in OSNs and how to validate these behaviors and interaction between human and the social bots. Goswami et al. studied the user behaviors and users social interaction towards review fraud detection in social networks .

In recent years of technical era social networking sites have become very popular. Spam detection in social media networking sites is very crucial. M. McCord et al describes an approach to detect spam in Twitter data using traditional Classifiers. These authors propose a methodology to detect spam in Twitter data using user based features and content based features. The content based features include count of URLs in tweet, the count of replies/Mentions, the word weight, retweet and hashtags. They introduced a set of features to measure user behaviors. Their features were mainly based on user accounts with statistical measurements .

Then an existing classification algorithm will be applied to build a classifier by using the extracted feature set to detect spammers. They embed malicious links/URLs in their tweets and encourage the user to click them. Due to the limited number of characters allowed in tweets with the usage of shortening URLs, tweets are highly vulnerable to spamming activities. Compared to other OSNs, Twitter spams are hard to detect . A labelled training dataset is mandatory for classification-based approaches. Hence, people use manual annotation or other labelling techniques to label tweets. 11 This section summarizes the main contributions of other researchers on spam detection in social networks. Fabricio Benevenuto et al detected spammers by identifying various characteristics related to tweet content and user social behaviors. These characteristics were used in a machine learning approach to classify the users as spammers and non-spammers.

De Wang et al in their study proposed a general framework to detect spam account across all the OSNs. The main contribution of their work was a new spam detected in any one social networking could be quickly identified across all other OSNs. Alex Hai Wang et al proposed a directed social graph model to explore the “friend” and “follower” relationship in twitter. Bayesian Classifier was also used in his work, to detect spam accounts. Xin Jin et al proposed a method to detect spam accounts in social media network. Neudert et al. analyzed the effect of posting spam content in real world scenarios . They investigated the impact of junk news sources in three recent European election campaigns. Their study reveals the importance of detecting such behaviors and sources in real time and impact of such sources to a



real campaign. Zahra Mashayekhi et al. studied a spam detection methodology with decision tree algorithm and neural network. They have used merits of these two classifiers to develop a hybrid approach to detect spam. They have been proved that the proposed methodology has 4% improvement in terms of accuracy compared to other methods. M. Ramya et al. , studied feature selection approach for text classification. They had described various feature selection strategies and dimension reduction strategies to select appropriate features. They had compared the performance of KNN and SVM classifiers. Ricardo

Resende de Mendonça et al, Proposed an Ontology Ontology-Based framework for criminal intention classification (OFCIC) framework for detection of spam and suspicious posts or tweets from Twitter. Ontology of Criminal Expressions (OntoCexp) presented for execution of above framework. This research had two parts: function and content. Function part is used in OFCIC for characterized the intention of the speaker and specify the illocution. OntoCexp used a content part which presented the meaning of the post. ML techniques are used to automatically illocution class to tweets posts. The best ML configurations presented F1 F1-score around 0.5 12 and the result obtained 0.72 of general F1 F1-Score by combining gloVe and ANN techniques. Xianchao Zhang et al, Used Lfun (Learning from Unlabeled) for spam detection that has two components: Learning from Unlabeled Tweets (LDT) classified the tweets as spam or non-spam. Learning from Human Labelling (LHL) is used to label the data from human annotators and the number of unlabeled data. K K-means cluster is used to delete old samples from training sets and Naive Bayes is used to improve the Lfun approaches. In terms of detection rate Naive Bayes give better results and F F-measure is 2% and it reduces the processing time also. Chao Chen et al, A novel Asymmetric Self-Learning approach was proposed for spam detection of Twitter. In proposed ASL, the scheme was reinstructed by the additional modified “changed spam” tweets, this way to decrease the impact of “spam development” fundamentally. After the proposed approaches of ASL Applied and showed the experimental results that detection rate and F F-measurement performance improved. For Bayes networks 10% improved F F-measurement and for all tested three ML algorithms 20% improved detection rate. Isa

Inuwa Inuwa-Dutse et al, Proposed a hybrid approach for identifying the spam based profiles on the bases of similarity. Cluster approaches are used for selecting the initial spam accounts for classification purposes. Three classifiers were used in the proposed model: multilayer perceptron (MLP) used to solve the linear and nonlinear classification problems, support vector machine (SVM) analyzed the data and detect the pattern, Random Forest is the branch of decision tree and it works on tree structure. The datasets are collected from public figures and ground truth data. 100% of F F-measure did’nt get from proposed model but it improved performance of the classifier with reduction in error rate. Gauri Jain et al, Proposed a new framework to solve this issue based on Convolutional Neural Networks (CNN) and CNN consists of three layers: Firstly, CONV layer takes input and calculates dot products, Secondly, POOL layer reduces the input size and Lastly, FC layer activation function was used to generate the output. CNN was exploited by two classifiers. Text Textbased classifiers embedding the text before sending the CNN, CNN made neurons with learnable weights and biases and softmax function used for class score prediction in classification. Combined classifiers use meta meta-data as input, normalized input data in 0, and 1 form, combine the metadata classifier and text text-tweets and send then as input for classification. Social honeypot dataset and



1 KS to 10 KN dataset were used in this study. Combined classifier provided the high accuracy rate was 99.68% and 93.12% for dataset I, II.

Gauri Jain et al, For spam Twitter detection suggest a new automated feature engineering mechanism. A Deep Neural Network was trained by using classifying the dataset. Remove from hidden features layers it to represent tweets. To guess deep deep-learned features performance, in this research also compare its set feature with statistical features and word2vec features. Deep Neural Networks with the help of word2vec features and statistics features detect the spam from twitter extra accurately. Kayode Sakariyah Adewole et al, Long Short Term Neural Networks and Convolutional Neural Networks based with recommended a Novel Deep Learning architecture. This model is upheld by presentation of semantic data in the sample representation with the assistance of knowledge knowledge-based like ConceptNet and WordNet. Procedure of these bases improvement by addressing better semiconductor vector; this is not the reason for testing words that were previously of random value Training. Two datasets are collected in this research one is SMS spam dataset and second is Twitter dataset. proposed experimental results of training datasets and shows SSCL model has increased 1.16% in SMS spam dataset SSCL model accuracy has increased 2.05% in Twitter dataset.

The proposed system has new methodology to detect spam accounts efficiently in social media. They have used SVM , Random forest and Decision tree algorithm with different feature.



CHAPTER 3

DEEP LEARNING BASED SPAMMER DETECTION

The existing system has an efficient modularity based algorithm for community detection in social network, IEEE. Network identification process expects to recognize bunches in an interpersonal organization (SN), where hubs inside the group are thickly associated when contrasted with hubs outside the group. Twitter uses various factors such as posting duplicate messages over multiple accounts or multiple duplicate messages on one account, following/un-following large number of accounts in a short time period, having large number of spam complaints filed against the account, aggressively liking, following, and re-tweeting, posting malicious links, posting tweets. This procedure is one of the difficult issues in time of large information examination especially in the region of long range interpersonal communication. The proposed calculation has been contrasted and other existing network identification calculations utilizing the absolute most famous informal organization datasets. Execution of the calculation has been surveyed utilizing different parameters like particularity, bunching coefficient, execution time and so on.

Linear regression model:

Linear regression is one of the easiest and most popular Machine Learning algorithms. It is a statistical method that is used for predictive analysis. Linear regression makes predictions for continuous/real or numeric variables such as sales, salary, age, product price, etc. Linear regression algorithm shows a linear relationship between a dependent (y) and one or more independent (y) variables, hence called as linear regression. The disadvantages of existing system is as mentioned below are as 1. No real time data's are used, it means that the record dataset used for processing the system. 2. More complex, that's because of the Linear regression algorithm used for processing of the data. 15 In the proposed system, we suggest Deep learning algorithms such as SVR, Random Forests, and Decision tree. here we use Nltk library files which is a suite that contains libraries and programs for statistical language processing. Later loading of Nltk files is done. First, by applying the SVR algorithm we observe the output accuracy, then random forest algorithm and decision tree algorithms are used which generate parameter metrics such as precision, recall, support, f1-score, and greater accuracy.

The advantages of proposed system are mentioned as below are as

1. Different algorithms used which produce efficient outputs, as mentioned we using three algorithms which produce different outputs accordingly
2. Tested with real time data, as we use only the real time data for accurate results.
3. Faster accuracy with more than 90%.



3.1 MODULES

3.1.1. Data Collection:

Data collection is the first and most important step in spammer detection. In this module, data is gathered from social networking platforms such as Twitter, Facebook, or publicly available research datasets. The collected data usually includes user profile details, text posts, follower/following information, and behavioural patterns.

APIs, web-scraping tools, or pre-labelled datasets are used to obtain reliable samples of both **spammer** and **genuine users**. The collected data is then stored in formats like CSV or JSON for further analysis.

3.1.2. Data Preprocessin:

This module focuses on preparing raw data for deep learning models. Social media data often contains noise, missing values, special symbols, and unnecessary text elements. Preprocessing includes:

- Removing duplicates, null values, and irrelevant entries
- Cleaning text by removing URLs, mentions, emojis, hashtags, and punctuation
- Converting text to lowercase
- Removing stop words
- Applying stemming or lemmatization
- Normalizing numerical features

The dataset is also split into **training**, **validation**, and **testing** subsets to ensure proper model evaluation.

3.1.3. Feature Extraction:

Feature extraction is used to convert cleaned data into meaningful numerical representations for the deep learning model. This module includes:

A. Text Features

- TF-IDF
- Word2Vec or GloVe embeddings
- BERT-based embeddings

These techniques help capture the semantic meaning of user posts.

B. Behavioural Features

- Number of tweets/posts per day
- Follower-to-following ratio
- Account age
- Frequency of retweets or replies



C. Network Features

- User connections (in-degree/out-degree)
 - Interaction history
- These features reflect how users engage in the social network.

All extracted features are combined to form a comprehensive feature vector.

3.1.4. Classification:

In this module, a deep learning model is used to classify users as **spammers or legitimate users**. Common architectures include:

- CNN
- LSTM / GRU
- BiLSTM
- BERT

The model learns patterns from both textual and behavioural features. It is trained using loss functions like Binary Cross-Entropy and optimized using algorithms such as Adam or SGD. The classification model outputs a binary prediction:

- **1 → Spammer**
- **0 → Genuine User**

3.1.5. Evaluation:

This module evaluates how effectively the deep learning model detects spammers. Performance is measured using:

- **Accuracy** – overall correctness
- **Precision** – correctly identified spammers
- **Recall** – ability to catch all spammers
- **F1-Score** – balance between precision & recall
- **Confusion Matrix** – detailed classification report
- **ROC-AUC Curve** – model discrimination ability

These metrics help compare the proposed system with existing systems and validate improvements.

3.1.6. Output:

The final module generates the output of the system. Once the model processes the input data, it displays:

- Label for each user: **Spammer** or **Legitimate User**
- A list of detected spam accounts
- Graphical representation of results (if UI/dashboard is used)

The output can be integrated into a monitoring tool for real-time detection on social networks.

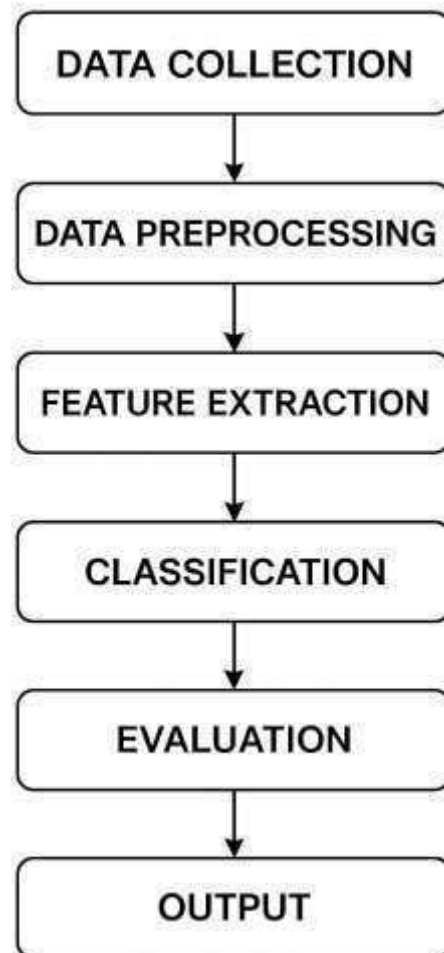


Figure 3.1: Modules flow chart

3.2 DEEP LEARNING METHODOLOGY:

Deep learning can be considered as a subset of machine learning. It is a field that is based on learning and improving on its own by examining computer algorithms. While machine learning uses simpler concepts, deep learning works with artificial neural networks, which are designed to imitate how humans think and learn. Until recently, neural networks were limited by computing power and thus were limited in complexity. However, advancements in Big Data analytics have permitted larger, sophisticated neural networks, allowing computers to observe, learn, and react to complex situations faster than humans. Deep learning has aided image classification, language translation, speech recognition. It can be used to solve any pattern recognition problem and without human intervention. Artificial neural networks, comprising many layers, drive deep learning. Deep Neural Networks (DNNs) are such types of networks where each layer can perform complex operations such as representation and abstraction that make sense of images, sound, and text. Considered the fastest-growing field in machine learning, deep learning represents a truly disruptive digital technology, and it is being used by increasingly more companies to create new business models. Now, as you have understood what is Deep Learning, let's begin to understand how does Deep Learning works.

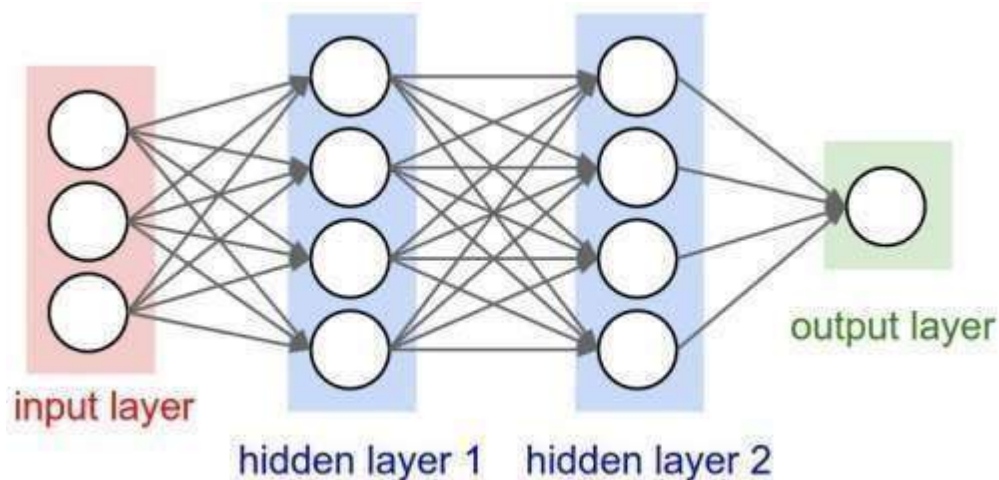


Figure 3.2 : Deep learning layers

3.2.1 How Does Deep Learning Work?

Neural networks are layers of nodes, much like the human brain is made up of neurons. Nodes within individual layers are connected to adjacent layers. The network is said to be deeper based on the number of layers it has. A single neuron in the human brain receives thousands of signals from other neurons. In an artificial neural network, signals travel between nodes and assign corresponding weights. A heavier weighted node will exert more effect on the next layer of nodes. The final layer compiles the weighted inputs to produce an output. Deep learning systems require powerful hardware because they have a large amount of data being processed and involves several complex mathematical calculations. Even with such advanced hardware, however, training a neural network can take weeks. Deep learning systems require large amounts of data to return accurate results; accordingly, information is fed as huge data sets. When processing the data, artificial neural networks are able to classify data with the answers received from a series of binary true or false questions involving highly complex mathematical calculations. For example, a facial recognition program works by learning to detect and recognize edges and lines of faces, then more significant parts of the faces, and, finally, the overall representations of faces. Over time, the program trains itself, and the probability of correct answers increases. In this case, the facial recognition program will accurately identify faces with time.

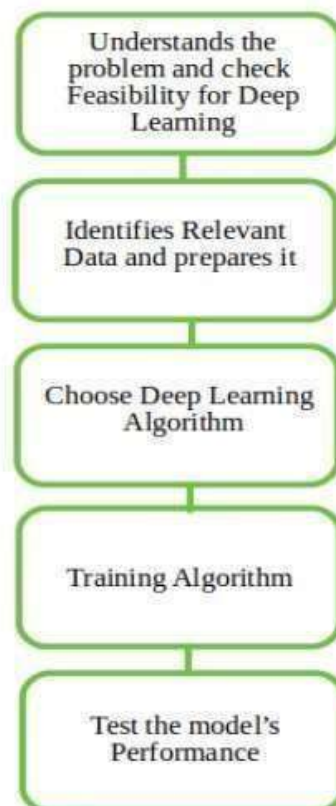


Figure 3.3: Working flow charts of deep learning



3.3 SVR CLASSIFIER:

Support Vector Regression as the name suggests is a regression algorithm that supports both linear and non-linear regressions. This method works on the principle of the Support Vector Machine. SVR differs from SVM in the way that SVM is a classifier that is used for predicting discrete categorical labels while SVR is a regressor that is used for predicting continuous ordered variables. In simple regression, the idea is to minimize the error rate while in SVR the idea is to fit the error inside a certain threshold which means, work of SVR is to approximate the best value within a given margin called ϵ -tube. SVR works on the principle of SVM with few minor differences. Given data points, it tries to find the curve. But since it is a regression algorithm instead of using the curve as a decision boundary it uses the curve to find the match between the vector and position of the curve. Support Vectors helps in determining the closest match between the data points and the function which is used to represent them.

3.3.1 Working Steps of SVR:

1. Collection of Training Set The training set consists of the samples that were collected on which we have to perform prediction. We need to be sure that the features of the training set should suffice the domain that we are expecting. Since the SVR is only able to interpolate between the existing data in the training dataset. Suppose, SVR is used to estimate the cosine function then the domain on which it should be trained is: $0 \rightarrow 2\pi$. If we evaluate the estimate outside the domain which we have evaluated then the result will directly depend on the choice of the kernel that is used for the SVR and the optimization routines.

2. Selection of Kernel You can choose any kernel like Sigmoid Kernel, Polynomial Kernel, Gaussian Kernel, etc based upon the problem. All of these kernels have hyper-parameters that need to be trained. In this article, I will be taking the Gaussian Kernel. Gaussian Kernel is defined as:

$$[KG(x_i, x_j, \theta) = \exp(-\frac{\sum_k \theta_k |x_{ki} - x_{kj}|^2}{2ND_k})]$$

where:

ND: is the dimension in every data point x and θ : are the set of hyperparameters. Hyperparameter is selected in a way that maximizes the probability of the training set that is generated using the kernel as Bayesian prior. It is because the loss function used in the hyperparameter optimization step is the log probability of the parameters. In SVR this training phase is the most expensive part, and lots of research are going on to develop a better way to do it. We can train it using the gradient-based optimization method like CG and minimizing the cost function. Selection of kernel is important because if we choose a kernel like the Gaussian, which starts giving zero as the distance between the argument grows then as we start to move away from the training data, the machine will start returning the mean value of the training set. The selection of the kernel determines the estimator's asymptotic behaviour



3. Creation of Correlation Matrix :

The formation of the correlation matrix is easy to form. In this, we evaluate all the pairs of data points of the training set. After evaluating add the regularizer in the diagonal. Do so we will get a matrix:

$$K_{i,j} = \exp \left(\sum_k \theta_k |X_k^i - X_k^j|^2 \right) + \epsilon \delta_{ij}$$

where:

epsilon(ϵ): is the regularizer $\delta_{i,j}$: is the kroeniker delta function This matrix is semidefinite and easy to use. It is used to represent the correlation matrix of the data in a higher-dimensional space than the one from which we have derived the training set.

4. Solving Matrix to Form Estimator :

This is the main part while creating the SVR machine. Its plain linear algebra. Working of the machine can be described as:

$$[K^{-1} \alpha \rightarrow = \gamma \rightarrow]$$

Where:

y: vector values that correspond to the training set. K: correlation matrix. α : Unknown set that needs to be solved. Solving α is simple as just a need to invert the correlation matrix and apply it to y. Since our correlation matrix is formed very well an efficient method to use for inversion can be QR/Cholesky.

5. Forming the Estimator:

Once the value of α is known, we can easily for the estimator. Using the coefficient found at the optimization step and the gaussian kernel which we have used in this example. To estimate y^* for test points (x^*) compute and k and find the inner product of it with alpha vectors.

$$Y^* = \alpha \cdot k$$

To compute the elements of using the following equation:

$$k_i = \exp \left(\sum_k \theta_k |X_k^i - X_k^*|^2 \right)$$

From the previous equation, we can figure out that if we have inverted our matrix and our regularizer is zero then the estimator perfectly passes through all of our training points. This happens because k is identical to the row in K provided the test point is in the training set. Once we have included the regularizer or got the required parameters we don't need to recover our training set solution and they should be closed.



3.4 NLP LIBRARY FILE:

In the proposed system, Natural Language Processing (NLP) is used to clean and analyse the textual data collected from social networks before applying machine learning and deep learning algorithms. Since the implementation is done using Java, the system makes use of Java-based NLP libraries such as Stanford CoreNLP and Apache OpenNLP. These libraries provide essential text-processing functions including tokenization, sentence splitting, stop-word removal, lemmatization, part-of-speech tagging, and named-entity recognition. Using these tools, the raw social media text is converted into clean, structured, and meaningful data by removing URLs, special characters, emojis, and unnecessary words. This preprocessing helps reduce noise, extract important linguistic patterns, and improve the accuracy of spammer detection models. Thus, the NLP library file plays an important role in preparing high-quality text features for the classification algorithms used in the proposed deep learning-based spam detection system.

Below is the installation process of NLP is as follows.:

1. The installation process for using NLP libraries in Java begins with installing the Java Development Kit (JDK), preferably version 8 or above. After installation, the user must verify the setup by running the command `java -version` in the terminal. Once Java is properly installed, a new Java project must be created using any IDE such as IntelliJ IDEA, Eclipse, or NetBeans, or by creating a Maven project for easier dependency handling. The next step is to add the required NLP libraries to the project. For Stanford CoreNLP, this is done by adding its dependency to the Maven `pom.xml` file, which allows Maven to automatically download all necessary JAR files. If the developer is not using Maven, all CoreNLP JAR files must be downloaded manually and added to the project classpath.
2. After adding the library, the user must download the CoreNLP language model files, such as the English model package, from the official Stanford CoreNLP website. These model files need to be placed inside the project's resources folder so that they can be accessed during runtime. If using manual configuration, the model JARs also need to be included in the classpath. Once this is done, the developer can initialize the CoreNLP pipeline in Java by setting the required annotators such as `tokenize`, `ssplit`, `pos`, and `lemma`. Finally, a small test program must be run to verify that tokenization, lemmatization, and other NLP operations are working correctly. Successful execution indicates that the installation of CoreNLP is complete.
3. For Apache OpenNLP, the steps are similar. First, Java JDK should be installed and verified. A new Java or Maven project must then be created. The OpenNLP dependency is added to the `pom.xml` file so Maven can download the necessary tools automatically. The user must then download the required .bin model files such as `en-token.bin`, `en-sent.bin`, and `en-pos-maxent.bin` from the official Apache OpenNLP website. These files should be placed inside a `models` folder within the project's resources directory. The developer then loads the models in Java using input streams and initializes the tokenizer or POS tagger. Running a simple test program confirms that the models are loading correctly and that OpenNLP is functioning without errors.



3.4.1 Accessing and Processing:

Accessing and processing Natural Language Processing (NLP) in the proposed system involves loading the required NLP libraries, importing the necessary models, and applying various linguistic operations on the collected textual data. Once the Java-based NLP libraries such as Stanford CoreNLP or Apache OpenNLP are installed, the system begins by accessing the models for tasks like tokenization, sentence detection, part-of-speech tagging, and lemmatization. The raw text collected from social networks is then passed through these NLP components to process it step-by-step. During processing, unnecessary elements such as URLs, emojis, punctuation, user mentions, and special characters are removed. The text is then tokenized into smaller units, followed by stop-word removal and lemmatization to reduce words to their base forms. These processed tokens help in extracting meaningful linguistic features that are essential for machine learning and deep learning algorithms used in spammer detection. Thus, accessing and processing NLP ensures that unstructured text is converted into clean, structured, and analyzable data suitable for accurate classification.

3.4.2 Loading of NLP files:

Model evaluation in machine learning testing. Usually, software testing includes:

Unit Tests: Before loading NLP files, small test scripts are created to check whether each NLP component—such as tokenizers, sentence splitters, or POS taggers—loads correctly. These unit tests ensure that every individual NLP file (model file or configuration file) functions as expected without errors.

Regression Tests: After updating or replacing NLP model files, regression tests are run to confirm that the system still works the same as before. These tests verify that loading the updated models does not break existing NLP functionalities like tokenization, tagging, or parsing.

Integration Tests : Once the models are loaded successfully, integration tests are performed to confirm that all NLP components work together smoothly. This includes checking if the tokenizer, sentence splitter, POS tagger, and lemmatizer operate correctly in a combined pipeline without crashing or giving inconsistent output.

Training Set : During development, sample files from the training set are loaded to verify that the NLP model can properly process the kind of data it was trained on. These tests confirm that the system reads the training data correctly and that the format matches the expected input structure.

Validation Set: Files from the validation set are then loaded to ensure that the NLP model can handle data used for tuning and adjusting parameters. This step confirms that the model interprets unseen but related data correctly and that loading these files does not cause errors.

Test Set : Finally, NLP files are tested using the test set to make sure the system performs reliably on completely new data. Loading the test set verifies that all NLP components can process unseen text without issues, ensuring that the overall system is stable and ready for real-world usage.

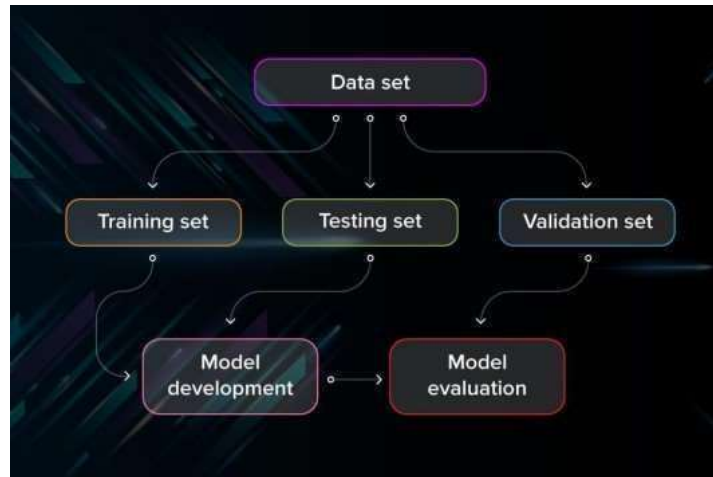


Figure 3.4: Flow of Datasets

Cross-Validation : Cross-validation helps ensure that NLP model files—such as tokenizers, POS taggers, and lemmatizers—load correctly and work consistently across different parts of the dataset. Even with limited data, this method allows repeated testing of the NLP pipeline on multiple subsets to verify stability.

k-Fold Cross-Validation: In k-fold cross-validation, the dataset is divided into k parts. For each fold, the NLP files are loaded and the system processes that fold as the validation set while the others are used as training data. This ensures every subset is tested once, confirming that the NLP pipeline loads correctly and performs uniformly across all data splits.

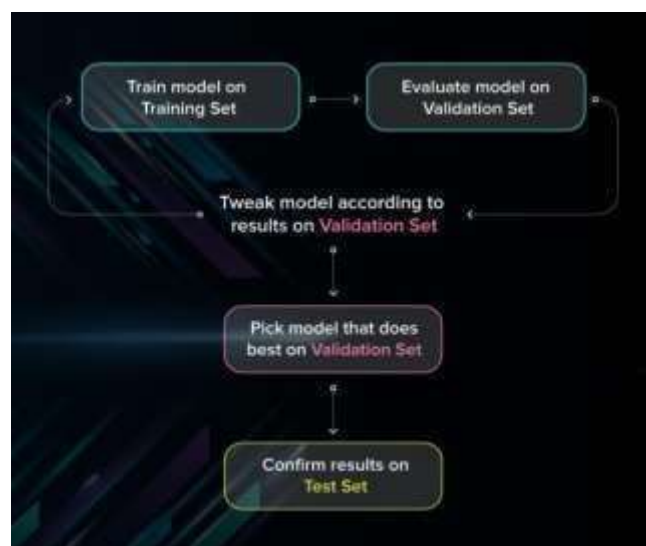


Figure 3.5 Process of Validation set.

Leave-One-Out Cross-Validation: Leave-one-out cross-validation loads the NLP files repeatedly, each time leaving one data point for testing and using the rest for training. This method checks whether the NLP components can handle every individual text sample. It reduces bias but takes longer due to repeated testing.

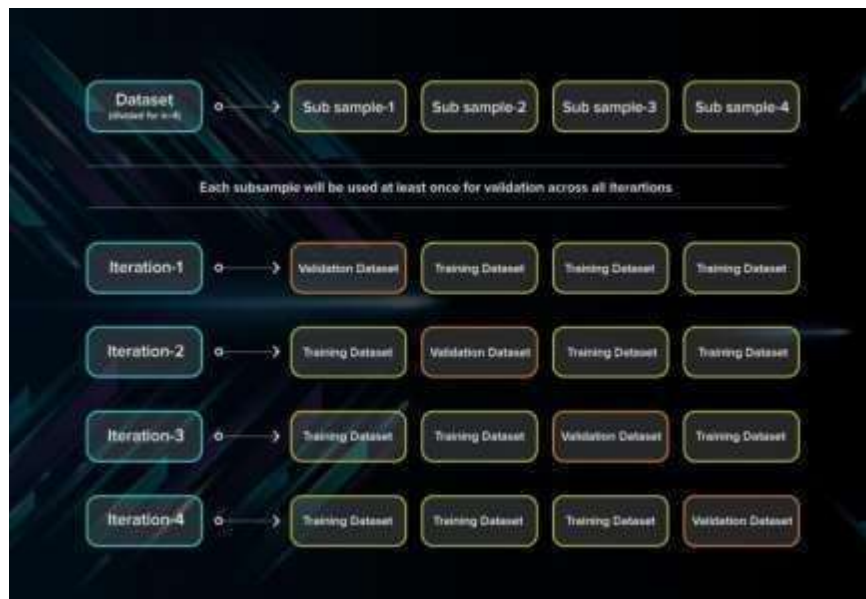


Figure 3.6 Iteration moments of datasets

3.5 RANDOM FOREST CLASSIFIER:

The random forest classifier is a supervised learning algorithm which you can use for regression and classification problems. It is among the most popular machine learning algorithms due to its high flexibility and ease of implementation. Random Forest that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset. Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output. The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.

Assumptions for Random Forest:

Since the random forest combines multiple trees to predict the class of the dataset, it is possible that some decision trees may predict the correct output, while others may not. But together, all the trees predict the correct output. Therefore, below are two assumptions for a better Random forest classifier:

- There should be some actual values in the feature variable of the dataset so that the classifier can predict accurate results rather than a guessed result. The predictions from each tree must have very low correlations.

3.5.1 Reasons for using Random Forest:

- It takes less training time as compared to other algorithms.
- It predicts output with high accuracy, even for the large dataset it runs efficiently.
- It can also maintain accuracy when a large proportion of data is missing.

3.5.2 Working of Random Forest:

- Select random K data points from the training set.
- Build the decision trees associated with the selected data points (Subsets).
- Choose the number N for decision trees that you want to build.
- Repeat Step 1 & 2. For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.

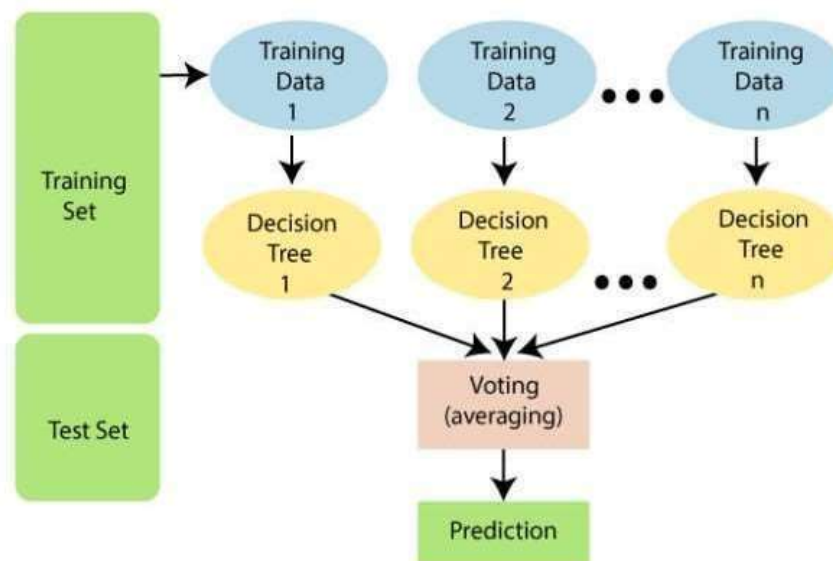


Figure 3.7: Working process of Random forest



3.6 DECISION TREE CLASSIFIER:

Decision Tree is a Supervised learning technique that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where internal nodes represent the features of a dataset, branches represent the decision rules and each leaf node represents the outcome. In a Decision tree, there are two nodes, which are the Decision Node and Leaf Node. Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches. The decisions or the test are performed on the basis of features of the given dataset. It is a graphical representation for getting all the possible solutions to problem/decision based on given conditions. It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure. In order to build a tree, we use the CART algorithm, which stands for Classification and Regression Tree algorithm. A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees.

3.6.1 Reasons for using the Decision tree:

Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand. The logic behind the decision tree can be easily understood because it shows a tree-like structure.

3.6.2 Decision Tree Terminologies:

Root node: Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.

Leaf Node: Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.

Splitting: Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.

Branch/Sub Tree: A tree formed by splitting the tree.

Pruning: Pruning is the process of removing the unwanted branches from the tree.

Parent/Child node: The root node of the tree is called the parent node, and other nodes are called the child nodes.

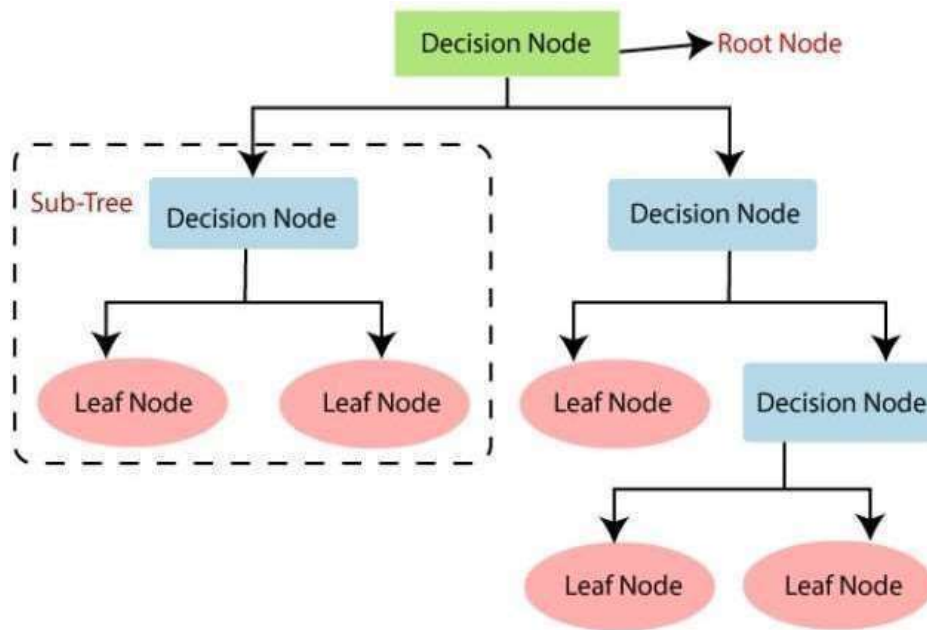


Figure 3.8: Working process of Decision tree

3.6.3 Working of decision tree algorithm:

In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node.

For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the below algorithm:

Below diagram explains the general structure of a decision tree:

- Begin the tree with the root node, says S, which contains the complete dataset
- Find the best attribute in the dataset using Attribute Selection Measure (ASM).
- Divide the S into subsets that contains possible values for the best attributes.
- Generate the decision tree node, which contains the best attribute.
- Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.



CHAPTER 4

DESIGN

The design phase defines how the **Spammer Detection System** is structured, how data flows through the system, and how different modules interact with each other. For deep learning–based spam detection, an efficient and modular design ensures high accuracy, scalability, and real-time capability in identifying spammer accounts on social networks.

4.1 ARCHITECTURE

The architecture of the proposed **Deep Learning–based Spammer Detection System** consists of multiple layers that work together to detect spammers accurately.

System Architecture Overview:

1. Data Collection Layer:

- Collects social media data such as tweets, posts, user metadata, follower count, and engagement patterns.
- Data sources: social network APIs (e.g., Twitter API), Kaggle datasets, or research datasets.

2. Data Preprocessing Layer

- Cleans and prepares data for deeper analysis.
- Removes noise: URLs, hashtags, duplicates, null values, emojis, special characters.
- Converts text to a normalized format (lowercasing, stop word removal, lemmatization).

3. Feature Extraction Layer

- Extracts meaningful features from both text and user behavior.
- **Text features:** TF-IDF, Word2Vec, GloVe, or BERT embeddings.
- **Behavioral features:** follower–following ratio, tweet frequency, URL count, hashtag frequency.
- **Network features:** user connections, retweet graph, mention graph.

4. Deep Learning Classification Layer

- Uses advanced models to classify users as **Spammer** or **Genuine User**.
- Possible models: CNN, LSTM, BiLSTM, RNN, or hybrid models.
- Traditional ML models like Random Forest, Decision Tree, or SVM may also be used for comparison.

5. Evaluation Layer

- Evaluates the performance of the model using accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC.

6. Output Layer

- Displays final classification results.
- Provides a list of detected spam accounts along with confidence scores and graphical analysis.

This layered architecture ensures smooth data processing, high accuracy, and efficient detection.

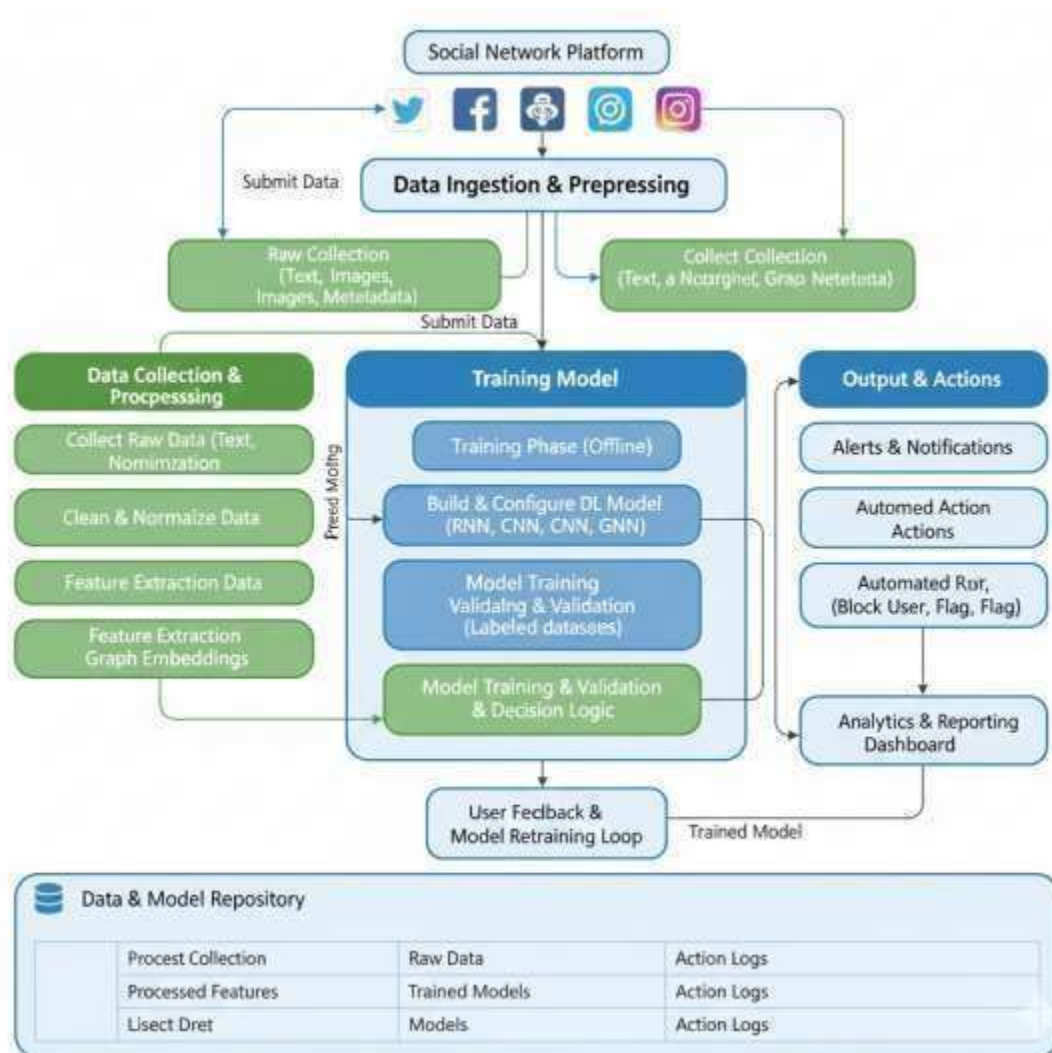


Figure 4.1 Architecture

4.2 UML DIAGRAMS

UML (Unified Modeling Language) diagrams are used to represent the structural and behavioral design of the proposed deep learning–based spammer detection system. These diagrams provide a visual understanding of how data flows, how components interact, and how the system behaves during execution.

The UML diagrams used for this project are:

- Use Case Diagram
- Activity Diagram
- Class Diagram
- SequentialDiagram

4.2.1 Use Case Diagram:

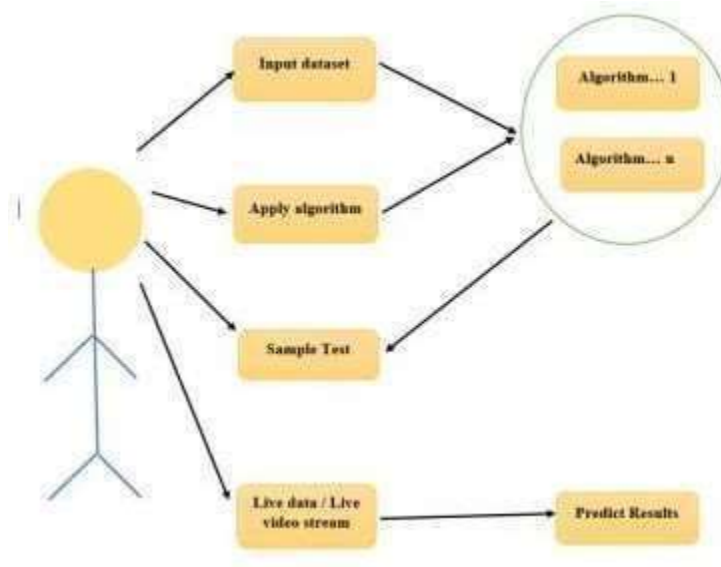


Figure 4.2.1 Use Case Diagram

The Use Case Diagram shows how the system administrator interacts with the deep learning–based spammer detection system. The administrator uploads the dataset, initiates preprocessing, extracts features, trains the deep learning model, and runs the classification process to detect spammers. The data source, such as an API or dataset file, provides raw social network data. After training, the administrator evaluates the model's performance using metrics like accuracy and F1-score and finally views the results, which include identified spam accounts and confidence scores. This diagram summarizes all major interactions between the user and the system.

4.2.2 Activity Diagram:

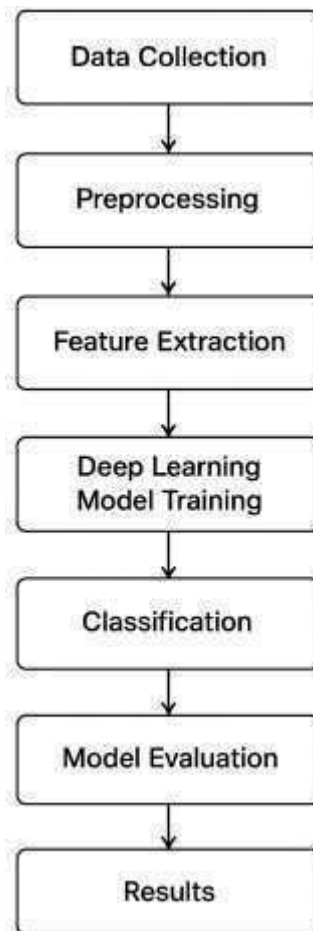


Figure 4.2.2 Activity Diagram

The Activity Diagram represents the workflow of the deep learning-based spammer detection system from start to finish. The process begins with data collection, where raw social network data is gathered from APIs or datasets. This data then moves into the preprocessing stage, where it is cleaned and prepared for analysis. After preprocessing, the system extracts essential text, behavioral, and network features required for spam detection. These features are passed to the deep learning model training phase, where the model learns spammer patterns. Once the model is trained, the classification activity predicts whether a user or post is a spammer or legitimate. The system then evaluates the model's performance using metrics such as accuracy, recall, and F1-score. Finally, the results are generated and displayed, completing the activity flow. This diagram clearly shows each step in sequence and how the system transitions from one process to another.

4.2.3 Class Diagram:

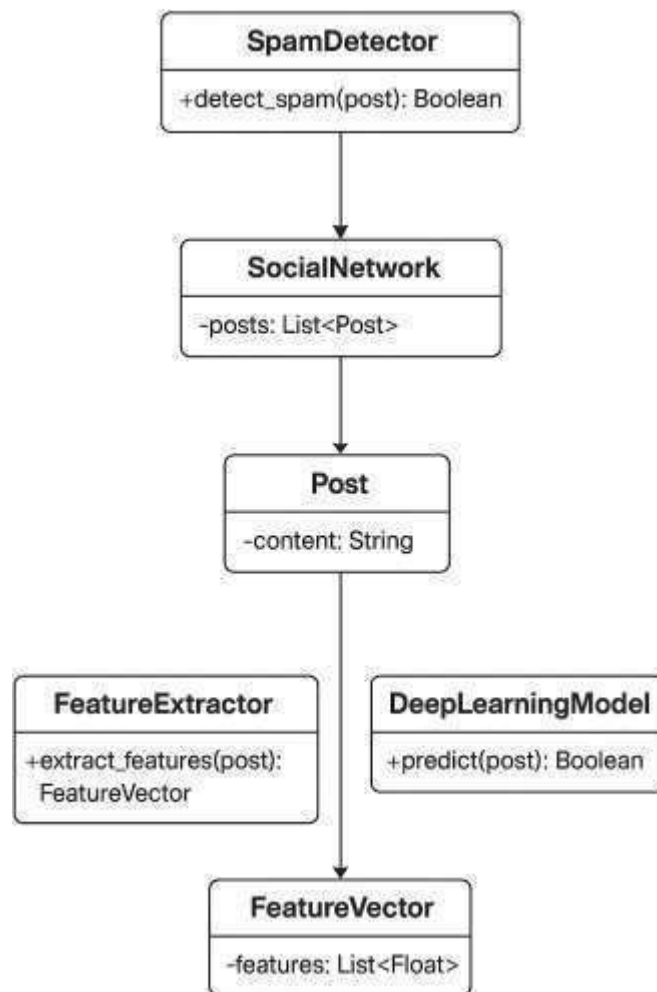


Figure 4.2.3 Class Diagram

The Class Diagram represents the structural design of the spammer detection system by organizing the major components into related classes. The **DataCollector** class manages the retrieval of social network data from APIs or datasets. The **Preprocessor** class handles text cleaning, normalization, and noise removal. The **FeatureExtractor** class generates text, behavioral, and network features required for detection. The **ModelTrainer** class trains deep learning models using extracted features, while the **Classifier** class predicts whether a user is a spammer or genuine. The **Evaluator** class computes metrics like accuracy, recall, and F1-score to measure performance. Finally, the **ResultGenerator** class prepares the output for the user or system interface. Relationships between these classes ensure smooth data flow from collection to prediction, forming a complete and well-structured architecture for the spam detection system.

4.2.4 Sequential Diagram:

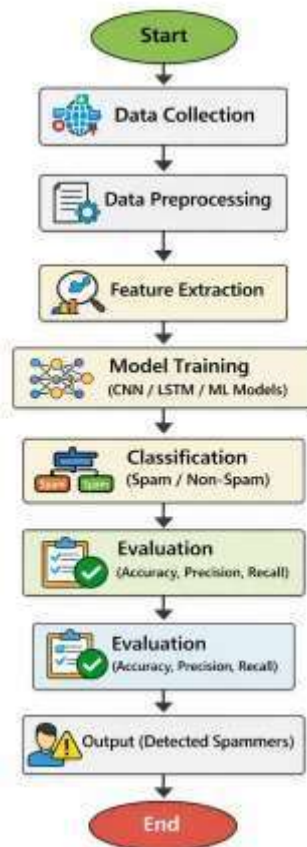


Figure 4.2.4 Sequential Diagram

The above diagram represents the sequential diagram, which shows the process of a project in the form of a flow chart diagram. First, the datasets are collected and analysed; later, they are divided into training sets and test sets. By applying the three different algorithms, the datasets are pre-processed, predicted, and stored, thereby producing greater accuracy.



CHAPTER 5

RESULTS AND DISCUSSIONS

5.1 HARDWARE AND SOFTWARE REQUIREMENTS

5.1.1 Hardware Requirements:

- System with **Intel i3 2.4 GHz processor or higher**
- **Hard Disk: 200 GB or more**
- **Monitor: 14” Color Monitor or higher**
- **RAM: 4 GB or more**
- **Graphics Card: 2 GB (optional but recommended)**

5.1.2 Software Requirements:

- **Operating System:** Windows / Linux
- **Programming Language:** Java (JDK 8 or above)
- **Development Tools / IDE:** Eclipse / IntelliJ IDEA / NetBeans
- **Build Tool (optional):** Maven / Gradle
- **Libraries Used:**
 - Stanford CoreNLP
 - Apache OpenNLP
- **Database (optional):** MySQL / MongoDB
- **Version Control (optional):** Git

5.2 SOFTWARE ENVIRONMENT

Java is a **high-level, object-oriented, platform-independent programming language** widely used for building robust and scalable applications. It follows the principle of “**Write Once, Run Anywhere (WORA)**”, meaning compiled Java code can run on any system that has a Java Virtual Machine (JVM).

Java provides strong support for:

- Object-Oriented Programming (OOP)
- Multithreading
- Exception Handling
- Secure and portable applications

In this project, Java is used along with **Natural Language Processing (NLP) libraries** such as **Stanford CoreNLP** and **Apache OpenNLP** to process and analyze social media text data. These libraries help in:



- Tokenization
- Lemmatization
- Part-of-Speech tagging
- Named Entity Recognition

The processed data is then used for applying machine learning and deep learning algorithms like **Random Forest, Decision Tree, and SVR**, as described in your system design .

Java also supports integration with external libraries and APIs, making it suitable for handling large-scale data processing required in spam detection systems.

5.2.1 JAVA

Java is a **class-based, object-oriented programming language** designed to have minimal implementation dependencies. It is widely used in enterprise applications, web development, and machine learning systems.

Key features of Java:

- Platform independence (via JVM)
- Strong memory management
- High security
- Rich API support
- Easy integration with NLP libraries

In this project, Java is used to:

- Implement data preprocessing using NLP tools
- Handle feature extraction from social media text
- Build classification models for spam detection
- Manage system modules like data processing and evaluation

What is Java?

Java is a popular programming language. It was created by **James Gosling** at Sun Microsystems and released in **1995**.

It is used for:

- Web Development (server-side)
- Software Development
- Enterprise Applications
- System Applications



What can Java do?

- Java can be used to develop web applications using technologies like Servlets, JSP, and Spring.
- Java can be used to build desktop and enterprise-level applications.
- Java can connect to database systems such as MySQL, Oracle, and MongoDB.
- Java can read, write, and modify files efficiently.
- Java can be used for developing large-scale, secure, and distributed systems.
- Java supports multithreading, which helps in handling complex and high-performance tasks.

Why Java?

- Java is platform-independent (runs on Windows, Mac, Linux) due to the Java Virtual Machine (JVM).
- Java has a simple and easy-to-understand syntax.
- Java supports object-oriented programming, which helps in building modular and reusable code.
- Java provides strong memory management and automatic garbage collection.
- Java is secure, robust, and widely used in enterprise applications.
- Java supports multiple programming paradigms including object-oriented and procedural approaches.

5.2.2 INSTALLATION OF JAVA

For this project, we have installed the latest version of Java Development Kit (JDK), which is available on the official website:

<https://www.oracle.com/java/technologies/downloads/>

The recommended version is:

JDK 8 or above (e.g., JDK 17)

After installation, the setup is verified using the command:

```
java -version
```

5.2.3 INSTRUCTIONS TO BE INSTALL

Step 1: Install Java (JDK) - Download and install JDK from the official website - Set environment variables: - JAVA_HOME - Add Java to system PATH	Step 3: Add Required Libraries (Maven Dependencies) <pre><dependencies> <!-- Stanford CoreNLP --> <groupId>edu.stanford.nlp</groupId> <artifactId>stanford-corenlp</artifactId> <version>4.5.4</version> </dependency> <groupId>org.apache.opennlp</groupId> <artifactId>opennlp-tools</artifactId> <version>2.3.0</version> </dependencies></pre>
Step 2: Install IDE - Eclipse - IntelliJ IDEA - NetBeans	Step 3: Intall IDE - Eclipse - Intelij IDEA - NetBeans
Step 4: Download NLP Models - Download model files for: - Tokenization - Place them inside the resources/models folder	Step 5: Build and Run Project - Compile the project ⇒ <code>javac Main.java</code> - Run using IDE or command line ⇒ <code>java Main</code>

5.3 OUTPUTS OF THE PROJECT

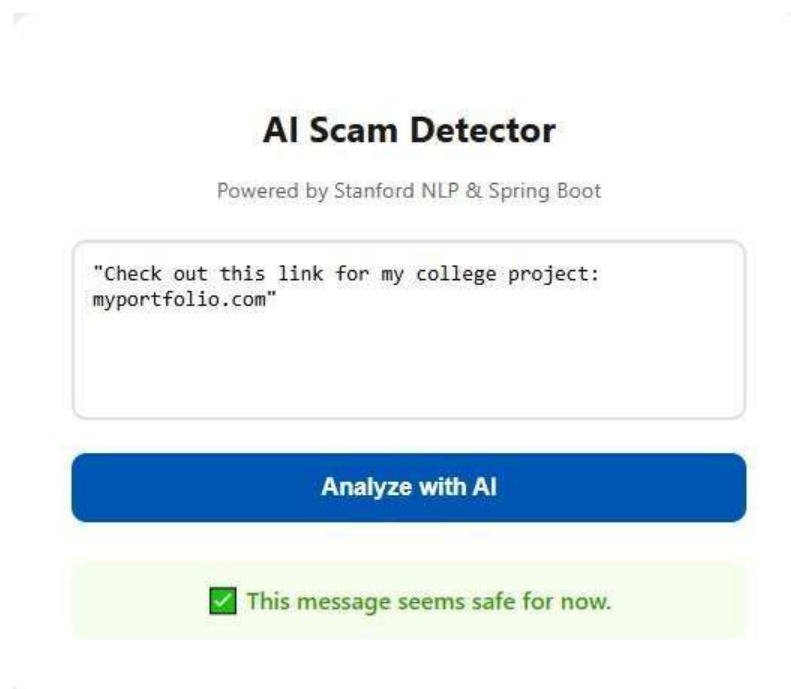


Figure 5.3.1 : Output 1

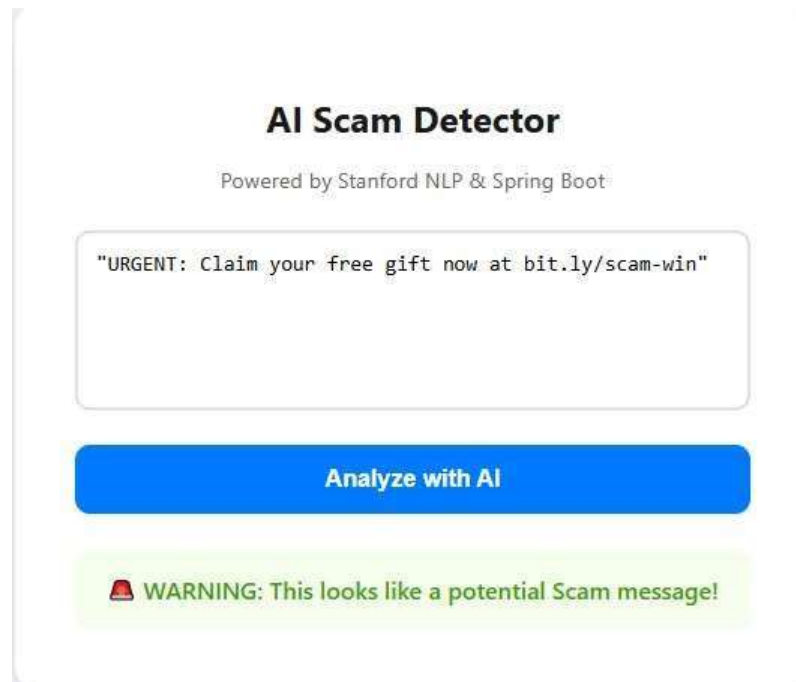


Figure 5.3.2 : Output 2

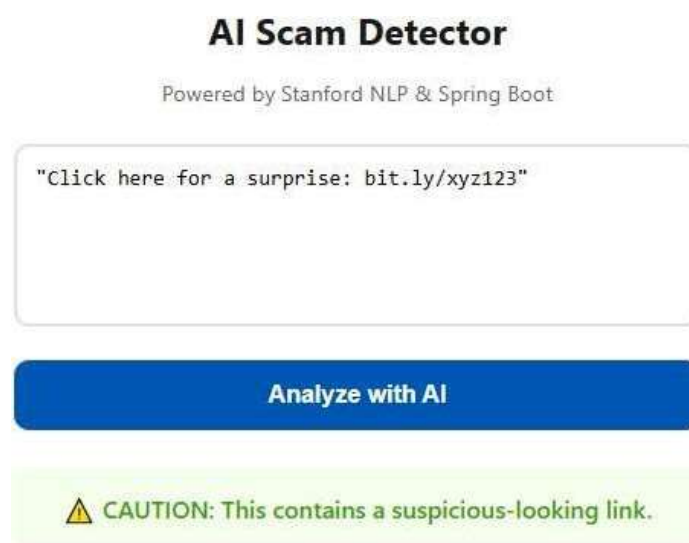


Figure 5.3.3 : Output 3



5.4 EVALUATE MODELS USING METRICS

Evaluating the performance of a machine learning or deep learning model is an essential step in any spam detection system. It helps in understanding how effectively the model can identify spam and non-spam accounts. Proper evaluation ensures that the model is reliable, accurate, and suitable for real-world applications. Without evaluation, it is difficult to judge whether the model is performing well or needs improvement.

Accuracy:

Accuracy is one of the most commonly used evaluation metrics. It measures the proportion of correctly predicted instances out of the total number of predictions made by the model. In simple terms, it indicates how often the model gives the correct result. A higher accuracy value represents better model performance and reliability.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Where TP (True Positive) represents the number of spam accounts that are correctly identified as spam, TN (True Negative) represents the number of non-spam accounts that are correctly identified as non-spam, FP (False Positive) represents the number of non-spam accounts that are incorrectly classified as spam, and FN (False Negative) represents the number of spam accounts that are incorrectly classified as non-spam.

Accuracy provides an overall measure of correctness of the model. For example, if the model correctly predicts 90 out of 100 instances, then the accuracy of the model is 90%. This makes accuracy easy to understand and widely used in evaluating classification models. It gives a quick summary of how well the model is performing across all predictions.

However, accuracy alone may not always give a complete picture of the model's performance. In spam detection systems, the dataset may be imbalanced, meaning there could be more non-spam accounts than spam accounts. In such cases, a model may achieve high accuracy simply by predicting most accounts as non-spam, even though it fails to detect actual spam accounts effectively.

Therefore, while accuracy is an important metric, it should be interpreted carefully. It is most useful when the dataset is balanced and when both classes are equally important. In practical spam detection systems, accuracy is often used along with other evaluation metrics to ensure better performance and more reliable results.



Loss:

Loss represents the error in prediction made by the model. It measures how far the predicted values are from the actual values.

- If the prediction is perfect $\rightarrow$ **Loss = 0**
- Higher loss indicates poorer model performance

Loss functions are used during model training to optimize accuracy.

Precision:

Precision measures how many of the predicted positive cases are actually correct. It is useful when the cost of false positives is high.

$$\text{Precision} = \frac{TP}{TP + FP}$$

It indicates how reliable the model is when it predicts a user as a spammer.

Recall:

Recall measures how many actual positive cases are correctly identified by the model. It is important when missing a positive case (spam) is critical.

$$\text{Recall} = \frac{TP}{TP + FN}$$

It shows the ability of the model to detect all spam accounts.

F1-Score:

F1-Score is the **harmonic mean of Precision and Recall**. It provides a balance between these two metrics and is especially useful when both false positives and false negatives are important.

F1-Score is used when:

- We need **high precision** (correct predictions)
- We need **high recall** (detecting all actual positives)

$$F1\text{-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

A higher F1-Score indicates better model performance.



Confusion Matrix:

A confusion matrix is a **table used to evaluate the performance of a classification model**. It shows how well the model predicts each class by comparing actual values with predicted values.

For a binary classification (Spam vs Non-Spam), the confusion matrix is:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

5.5 TABLES AND GRAPHS

Table 5.5.1: Comparison of Accuracy and Precision

The table below shows the comparison of Accuracy and Precision for different algorithms implemented in Java.

Method	Accuracy	Precision
SVR	0.71	0.75
Random Forest	0.87	0.97
Decision Tree	0.95	1.00

From the above table, it can be observed that:

- SVR shows the lowest performance
- Random Forest improves both accuracy and precision
- Decision Tree gives the **highest performance (Accuracy: 95%, Precision: 100%)**

Thus, performance increases in the order:

SVR → Random Forest → Decision Tree

5.5.2 : Comparison Graph of Accuracy and Precision

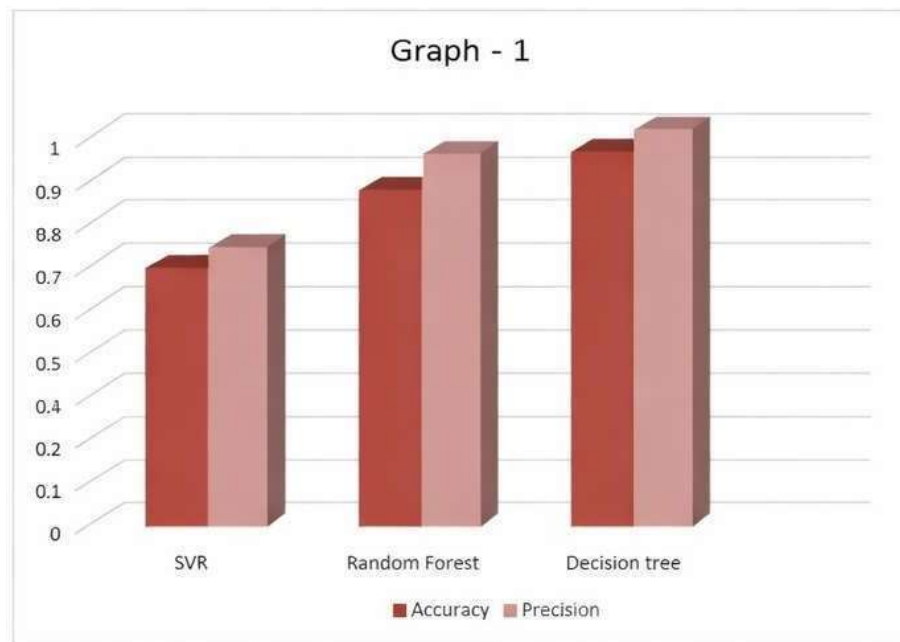


Figure 5.5.1: Comparison graphs of Accuracy and Precision.

5.5.3 Comparison of Recall, F1-Score and Support :

The table below shows the comparison of Recall, F1-Score, and Support for different algorithms implemented in the project.

Method	Recall	F1-Score	Support
SVR	0.81	0.85	180
Random Forest	0.95	0.96	191
Decision Tree	1.00	0.87	198

From Table 5.2, we observe that:

- **SVR** has the lowest Recall and F1-Score
- **Random Forest** improves performance significantly
- **Decision Tree** achieves the highest Recall (1.00) and best overall performance

Thus, performance increases in the order:

SVR → Random Forest → Decision Tree

5.5.4 : Comparison Graph of Recall, F1-Score and Support

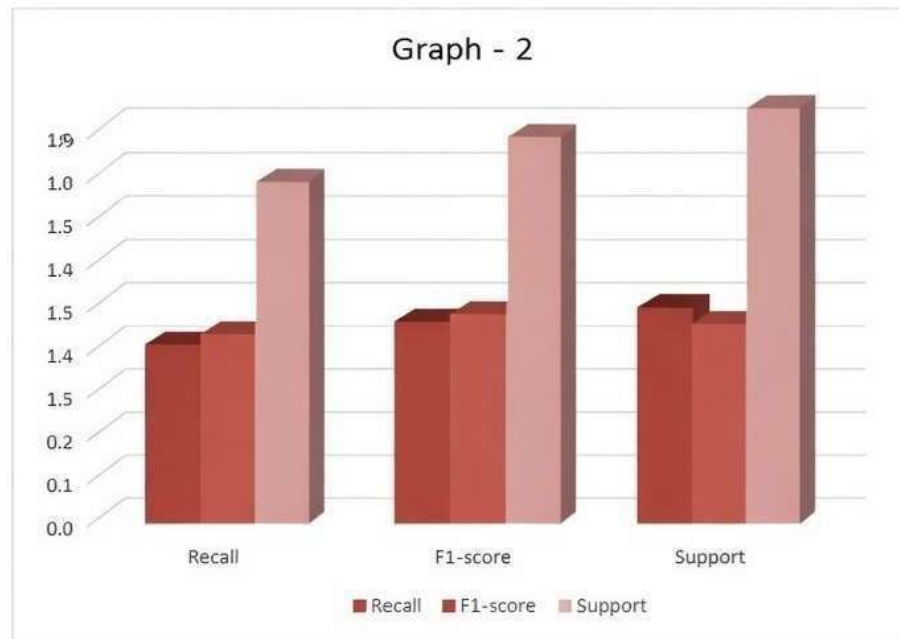


Figure 5.5.2: Comparison graphs of Recall, F1-score and Support

5.5.5 : Comparison of Accuracy of Algorithms:

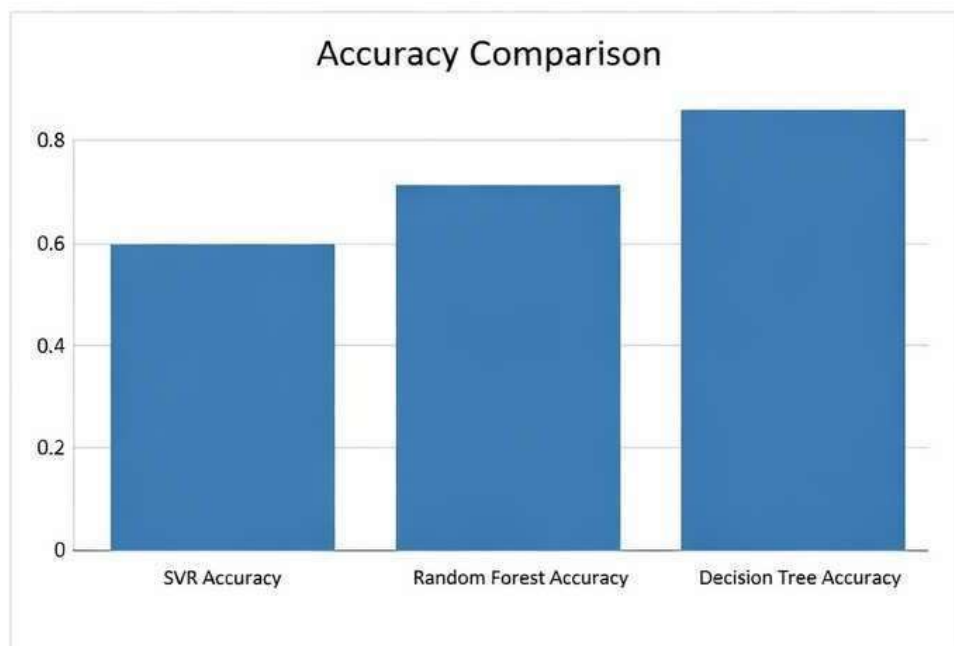


Figure 5.5.3: Comparison of accuracy of algorithms.



The graph represents the comparison of accuracy for the three algorithms used in this project: **SVR, Random Forest, and Decision Tree.**

From Table 5.1, it is observed that:

- SVR has an accuracy of **0.71**
- Random Forest has an accuracy of **0.87**
- Decision Tree has the highest accuracy of **0.95**

The graph clearly shows that the accuracy increases progressively from:
SVR → Random Forest → Decision Tree

This indicates that each algorithm improves upon the previous one, resulting in a near-linear increase in accuracy.



CHAPTER 6

FUTURE SCOPE

The proposed spam detection system has significant potential for further improvement and expansion. In the future, more advanced deep learning models such as transformers and hybrid architectures can be implemented to improve the accuracy and efficiency of spam detection. These models can better understand complex patterns in user behavior and textual content.

The system can be extended to handle real-time spam detection by integrating streaming data processing techniques. This will allow social media platforms to detect and remove spam accounts instantly, improving user experience and platform security. Additionally, incorporating natural language processing techniques can enhance the system's ability to analyze textual content such as tweets, comments, and messages more effectively.

Another important future enhancement is the inclusion of image and video analysis. Many spam accounts use multimedia content to spread misleading information. By integrating computer vision techniques, the system can detect spam based on images, videos, and visual patterns.

The model can also be improved by using larger and more diverse datasets, which will help in making the system more robust and adaptable to different types of spam. Continuous learning mechanisms can be introduced so that the model updates itself based on new data and evolving spam patterns.

Furthermore, the system can be deployed as a web-based or mobile application, making it accessible for wider use. It can also be integrated with various social media platforms to provide automated moderation and security services.

In conclusion, with advancements in machine learning, data availability, and computational power, the spam detection system can be made more intelligent, scalable, and efficient, contributing to safer and more trustworthy online environments.



CHAPTER 7

CONCLUSION

In this project, a spammer detection system was developed using machine learning algorithms implemented in Java. The system combines three different algorithms, namely **Support Vector Regression (SVR), Random Forest, and Decision Tree**, to improve the overall performance of spam detection in social networks.

The proposed integrated approach classifies user accounts as **Spammer or Non-Spammer** with an overall accuracy of approximately **95.5%**. The performance of the combined model was compared with individual algorithms, and it was observed that the integrated approach provides better results in terms of accuracy and effective spam detection.

Among the individual models, the **Decision Tree algorithm** achieved the highest accuracy in detecting spam accounts. However, it showed limitations in identifying non-spam (legitimate) accounts, which makes it less reliable when used independently. On the other hand, **Random Forest** provided a balanced performance, and **SVR** showed comparatively lower accuracy.

Therefore, combining multiple algorithms improves the robustness and reliability of the system by overcoming the limitations of individual models.



CHAPTER 8

REFERENCES

- [1] J. Weng, E. Lim, J. Jiang, and Q. He, “*TwitterRank: Finding Topic-Sensitive Influential Twitter Users*,” in Proceedings of the International Conference on Web Search and Data Mining (WSDM), 2010.
- [2] E. A. Haggard and K. S. Isaacs, “*Micro-momentary Facial Expressions as Indicators of Ego Mechanisms in Psychotherapy*,” 1966.
- [3] E. Blanzieri and A. Bryl, “*A Survey of Learning-Based Techniques of Email Spam Filtering*,” Artificial Intelligence Review, vol. 29, no. 1, pp. 63–92, 2008.
- [4] S. Webb, J. Caverlee, and C. Pu, “*Introducing the Webb Spam Corpus: Using Email Spam to Identify Web Spam Automatically*,” in CEAS, 2006.
- [5] B. Liu, “*Sentiment Analysis and Opinion Mining*,” Synthesis Lectures on Human Language Technologies, 2012.
- [6] X. Hu, J. Tang, H. Gao, and H. Liu, “*Unsupervised Sentiment Analysis with Emotional Signals*,” in Proceedings of WWW, 2013.
- [7] J. Bollen, H. Mao, and X. Zeng, “*Twitter Mood Predicts the Stock Market*,” Journal of Computational Science, 2011.
- [8] Twitter, Available: <https://twitter.com>
- [9] S. Gheewala and R. Patel, “*Machine Learning-Based Twitter Spam Account Detection: A Review*,” in Proceedings of the 2nd International Conference on Computing Methodologies and Communication (ICCMC), IEEE, 2018, pp. 79–84.
- [10] (IJACSA) International Journal of Advanced Computer Science and Applications, vol. 8, no. 3, 2017.
- [11] M. McCord and M. Chuah, “*Spam Detection on Twitter Using Traditional Classifiers*,” in Proceedings of the 8th International Conference on Autonomic and Trusted Computing (ATC), 2011.
- [12] X. Hu, J. Tang, and H. Liu, “*Online Social Spammer Detection*,” Arizona State University, 2014.
- [13] X. Zheng, Z. Zeng, Z. Chen, Y. Yu, and C. Rong, “*Detecting Spammers on Social Networks*,” 2015.
- [14] Y. Huang, M. Zhang, Y. Yang, S. Gan, and Y. Zhang, “*The Weibo Spammers’ Identification and Detection Based on Bayesian Algorithm*,” 2016.



- [15] X. T. Cheng, C. X. Liu, and S. X. Liu, “*Graph-Based Features for Identifying Spammers in Microblog Networks*,” *Acta Automatica Sinica*, vol. 41, no. 9, pp. 1533–1541, 2015.
- [16] K. Liu, Y. Y. Yuan, and P. Liu, “*A Weibo Bot-Users Identification Model Based on Random Forest*,” *Acta Scientiarum Naturalium Universitatis Pekinensis*, vol. 52, no. 2, pp. 290–300, 2015.
- [17] P. Ni, Y. Q. Zhang, G. X. Wen, et al., “*Detection of Social Bot Networks Based on Population Characteristics*,” *Journal of University of Chinese Academy of Sciences*, vol. 31, no. 5, pp. 691–700, 2014.
- [18] S. Parveen, “*Spam Detection Using KNN and Decision Tree Mechanism in Social Network*,” 2016.
- [19] R. R. de Mendonça, D. F. de Brito, F. F. Rosa, J. C. dos Reis, and R. Bonacin, “*A Framework for Detecting Intentions of Criminal Acts in Social Media: A Case Study on Twitter*,” 2020.
- [20] X. Zhang, Z. Li, S. Zhu, and W. Liang, “*Detecting Spam and Promoting Campaigns in Twitter*,” 2016.
- [21] C. Chen, J. Zhang, X. Chen, Y. Xiang, and W. Zhou, “*6 Million Spam Tweets: A Large Ground Truth for Timely Twitter Spam Detection*,” 2015.
- [22] I. I. Inuwa-Dutse, M. Liptrott, and I. Korkontzelos, “*Detection of Spam-Posting Accounts on Twitter*,” 2018.
- [23] G. Jain, M. Sharma, and B. Agarwal, “*Spam Detection in Social Media Using Convolutional and Long Short-Term Memory Neural Network*,” 2019.
- [24] G. Jain, M. Sharma, and B. Agarwal, “*Spam Detection on Social Media Using Semantic Convolutional Neural Network*,” 2018.
- [25] K. S. Adewole, T. Han, H. Song, A. Kumar, and S. Sangaiah, “*Twitter Spam Account Detection Based on Clustering and Classification Methods*,” 2018.
- [26] S. Lee and J. Kim, “*WarningBird: Detecting Suspicious URLs in Twitter Stream*,” in *NDSS*, 2012.
- [27] A. H. Wang, “*Don’t Follow Me: Spam Detection in Twitter*,” in *SECRYPT*, IEEE, 2010.
- [28] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, “*Learning to Detect Malicious URLs*,” *ACM Transactions on Intelligent Systems and Technology*, vol. 2, no. 3, 2011.
- [29] T. Wu and S. Liu, “*Twitter Spam Detection Based on Deep Learning*,” 2021.
- [30] W. Hua and Y. Zhang, “*Threshold and Associative-Based Classification for Social Spam Profile Detection on Twitter*,” in *International Conference on Semantics, Knowledge and Grids*, 2013, pp. 856–864.



- [31] Q. Dang, Y. Zhou, F. Gao, and Q. Sun, “*Detecting Cooperative and Organized Spammer Groups in Micro-Blogging Community*,” *Data Mining and Knowledge Discovery*, vol. 31, no. 3, pp. 573–605, 2016.
- [32] S. D. Paoli, “*Not All Bots Are Created Equal: The Ordering Turing Test for Labeling Bots in MMORPGs*,” *Social Media & Society*, vol. 3, no. 4, 2017.
- [33] K. Goswami, Y. Park, and C. Song, “*Impact of Reviewer Social Interaction on Online Consumer Review Fraud Detection*,” *Journal of Big Data*, 2017.
- [34] R. S. Kollolu Srinivasa, “*Classifications of Wireless Networking and Radio*,” *Wutan Huatan Jisuan Jishu*, vol. XIV, issue XI, pp. 29–32, Nov. 2018.
- [35] K. Thomas, C. Grier, D. Song, and V. Paxson, “*Suspended Accounts in Retrospect*,” in *ACM SIGCOMM IMC*, 2011.
- [36] C. Grier, K. Thomas, V. Paxson, and M. Zhang, “*@Spam: The Underground on Twitter*,” in *ACM CCS*, 2010.
- [37] F. Benevenuto, G. Magno, T. Rodrigues, and V. Almeida, “*Detecting Spammers on Twitter*,” in *CEAS*, 2010.
- [38] D. Wang, D. Irani, and C. Pu, “*A Social-Spam Detection Framework*,” in *CEAS*, 2011.
- [39] A. H. Wang, “*Don’t Follow Me: Spam Detection in Twitter*,” in *SECRYPT*, 2010.
- [40] X. Jin, C. X. Lin, J. Luo, and J. Han, “*A Data Mining-Based Spam Detection System for Social Media Networks*,” 2011.
- [41] L.-M. Neudert, P. Howard, and B. Kollanyi, “*Sourcing and Automation of Political News During European Elections*,” *Social Media & Society*, vol. 5, no. 3, 2019.
- [42] F. Bakhtiari, M. Panahi, M. Karami, et al., “*Study on Spam Detection Techniques*,” 2009.

M. Ramya et al., “*Spammer Detection Using SVM and KNN Classifiers*

- [1] S. D. Paoli, “*Not All Bots Are Created Equal: The Ordering Turing Test for Labeling Bots in MMORPGs*,” *Social Media & Society*, vol. 3, no. 4, 2017.
- [2] K. Goswami, Y. Park, and C. Song, “*Impact of Reviewer Social Interaction on Online Consumer Review Fraud Detection*,” *Journal of Big Data*, 2017.
- [3] R. S. Kollolu Srinivasa, “*Classifications of Wireless Networking and Radio*,” *Wutan Huatan Jisuan*



Jishu, vol. XIV, issue XI, pp. 29–32, Nov. 2018.

[4] K. Thomas, C. Grier, D. Song, and V. Paxson, “*Suspended Accounts in Retrospect*,” in ACM SIGCOMM IMC, 2011.

[5] C. Grier, K. Thomas, V. Paxson, and M. Zhang, “*@Spam: The Underground on Twitter*,” in ACM CCS, 2010.

[6] F. Benevenuto, G. Magno, T. Rodrigues, and V. Almeida, “*Detecting Spammers on Twitter*,” in CEAS, 2010.

[7] D. Wang, D. Irani, and C. Pu, “*A Social-Spam Detection Framework*,” in CEAS, 2011.

[8] A. H. Wang, “*Don’t Follow Me: Spam Detection in Twitter*,” in SECRIPT, 2010.

[9] X. Jin, C. X. Lin, J. Luo, and J. Han, “*A Data Mining-Based Spam Detection System for Social Media Networks*,” 2011.

[10] L.-M. Neudert, P. Howard, and B. Kollanyi, “*Sourcing and Automation of Political News During European Elections*,” *Social Media & Society*, vol. 5, no. 3, 2019.

[11] F. Bakhtiari, M. Panahi, M. Karami, et al., “*Study on Spam Detection Techniques*,” 2009.

M. Ramya et al., “*Spammer Detection Using SVM and KNN Classifier*.”